UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA

Verificador de Integridad de Paquetes CRC-8 con Buffer RAM y Decodificador de Direcciones para Redes de Sensores IoT

Villafuerte, Hernán

CURSO:

CIRCUITOS DIGITALES

INTEGRANTES:

Alarcon Consepcion, Genesis Elizabeth

Bolivar Morales, Jerson Franco

Cuyutupa Pereda, Juana Milagro

Sánchez Ramirez Allison Blanca Yarin

Lima, 2026

1. Planteamiento del Problema

En las redes de sensores IoT (Internet of Things), múltiples nodos transmiten datos de telemetría —temperatura, humedad, presión— hacia un nodo concentrador a través de enlaces serie (UART). Durante la transmisión, los datos están expuestos a interferencias electromagnéticas, ruido eléctrico y problemas de sincronización que pueden corromper uno o más bits del paquete, generando lecturas erróneas que comprometen la integridad del sistema.

El estándar de verificación de redundancia cíclica CRC-8, ampliamente utilizado en protocolos como I2C, 1-Wire e industriales, permite detectar hasta el 100 % de errores de un solo bit y una alta proporción de errores en ráfaga mediante un polinomio generador aplicado sobre los bytes del paquete. Sin embargo, su implementación a nivel de hardware digital —con circuitos combinacionales y secuenciales— es materia de estudio en el nivel formativo de la ingeniería de telecomunicaciones.

El presente proyecto propone diseñar e implementar un sistema digital verificador de integridad de paquetes basado en CRC-8 que reciba bytes por UART, los almacene en un buffer de memoria RAM con decodificador de direcciones, calcule el CRC mediante un registro de desplazamiento LFSR y emita una señal indicadora de error o trama válida. El sistema evidenciará la arquitectura FSM + Decodificador de Direcciones + Memoria (RAM), conforme a la sumilla del curso.

2. Objetivos

2.1 Objetivo General

Diseñar e implementar en Arduino un sistema digital verificador de integridad de paquetes de datos basado en el algoritmo CRC-8, que incorpore una FSM de recepción, un decodificador de direcciones y un buffer de memoria RAM, aplicable a nodos receptores en redes de sensores IoT.

2.2 Objetivos Específicos

1. OE1: Diseñar la máquina de estados finitos (FSM) de recepción UART que coordine la captura byte a byte del paquete y la escritura ordenada en el buffer RAM.
2. OE2: Implementar el decodificador de direcciones que traduzca el contador de escritura/lectura en señales de selección de posición en el buffer RAM (sel[3:0]).
3. OE3: Desarrollar el módulo de cálculo CRC-8 mediante un registro de desplazamiento LFSR con realimentación (polinomio 0x07) y verificar el resultado contra el byte de control recibido.
4. OE4: Medir y analizar las tres métricas de desempeño definidas contrastando valores teóricos con resultados experimentales obtenidos en el prototipo físico.
5. OE5: Documentar el diseño completo en formato de artículo IEEE, incluyendo diagramas de bloques, diagramas ASM, código fuente comentado y análisis de resultados.

3. Diagrama de Bloques Funcional

La Figura 1 presenta la arquitectura del sistema propuesto por el equipo, elaborada como diagrama conceptual de alto nivel (obtenido de la propuesta inicial). La Figura 2 muestra la arquitectura formal con los cinco bloques funcionales interconectados que conforman el sistema.

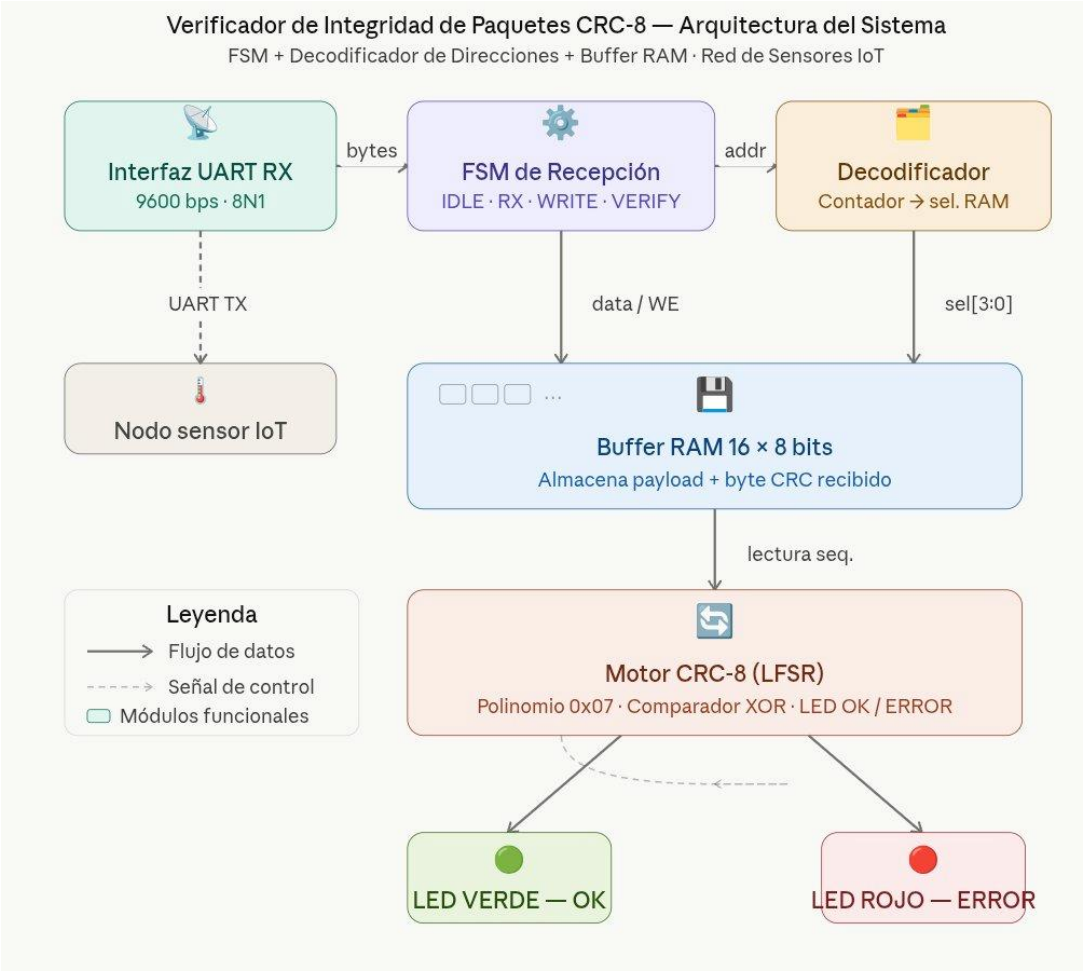

Figura 1. Diagrama conceptual de la arquitectura CRC-8 (elaboración del equipo).

El flujo de operación es el siguiente: (1) el nodo IoT transmite bytes por UART y la FSM transita al estado RX; (2) cada byte recibido activa el decodificador de direcciones, que selecciona la celda RAM correspondiente según el contador de escritura; (3) al completar el paquete la FSM transita a VERIFY, lee la RAM secuencialmente y alimenta el motor CRC-8 (LFSR); (4) el resultado se compara con el byte CRC al final del paquete y se activa LED VERDE (OK) o LED ROJO (ERROR).

Figura 2. Descripción de los bloques funcionales del sistema.

Bloque	Componente / Módulo	Función principal
Interfaz UART RX	UART 9600 bps, 8N1	Captura bytes serie del nodo sensor IoT y los entrega en paralelo a la FSM.
FSM de Recepción	Estados: IDLE, RX, WRITE, VERIFY	Unidad de control central: coordina la captura de bytes, la escritura en RAM y el inicio de la verificación CRC.

Bloque	Componente / Módulo	Función principal
Decodificador de Direcciones	Lógica combinacional, contador → sel[3:0]	Convierte el valor del contador de escritura/lectura en señales de selección de la celda RAM activa.
Buffer RAM 16×8 bits	Memoria SRAM simulada; bus D[7:0], A[3:0], WE	Almacena secuencialmente los bytes del payload más el byte CRC recibido al final del paquete.
Motor CRC-8 (LFSR)	LFSR 8-bit, polinomio 0x07, comparador XOR	Lee la RAM byte a byte, calcula el CRC-8 con realimentación LFSR y compara con el byte CRC recibido. Emite señal OK o ERROR.

4. Plataforma Seleccionada y Justificación

Se utilizará Arduino Uno/Mega como plataforma de implementación, con simulación previa en Wokwi. La elección se justifica por:

- Disponibilidad inmediata: El equipo cuenta con acceso a placas Arduino sin necesidad de reserva de laboratorio adicional.
- Facilidad de prototipado: La implementación en C++ permite modelar la FSM, el decodificador y el buffer RAM mediante estructuras de control equivalentes a los circuitos digitales del curso.
- Medición integrada: Las funciones micros() y millis() permiten medir latencia con precisión de microsegundos sin instrumentación adicional.
- Simulación gratuita: Wokwi soporta dos Arduinos interconectados por UART, lo que permite validar la lógica antes de la implementación física.
- Escalabilidad a FPGA: El diseño es portable a FPGA DE2-115 (Verilog) si el docente lo requiere, ya que la arquitectura FSM + Decodificador + RAM es independiente de la plataforma.

5. Métricas de Rendimiento Propuestas

Se definen tres métricas cuantitativas alineadas con los estándares de evaluación del curso, medibles experimentalmente y contrastables con valores teóricos:

N°	Métrica	Descripción	Método de medición	Valor teórico esperado
M1	Porcentaje de detección de errores (%)	Relación entre errores inyectados (bits corruptos) y errores detectados por el CRC-8.	Enviar 100 paquetes con 1 bit corrompido en posiciones aleatorias; contar detecciones correctas.	100% para errores de 1 bit (garantía del CRC-8)
M2	Latencia de verificación (µs)	Tiempo desde recepción del último byte hasta la activación de la señal OK/ERROR.	micros() antes y después del bloque de cálculo CRC en el Arduino receptor.	< 500 µs para paquetes de 8 bytes a 9600 bps
M3	Throughput máximo (bytes/s)	Velocidad máxima de procesamiento de paquetes completos sin pérdida de datos ni fallo de verificación.	Incrementar baudrate progresivamente (9600, 19200, 38400, 57600 bps) y registrar el punto de fallo.	> 960 bytes/s a 9600 bps (sin overhead de verificación)

Tabla 1. Métricas de rendimiento propuestas para el sistema verificador CRC-8.

6. Referencias

- qR. Williams, 'A Painless Guide to CRC Error Detection Algorithms,' Rocksoft Pty Ltd, 1993. [En línea]. Disponible en: https://www.ross.net/crc/download/crc_v3.txt
- Microchip Technology, 'ATmega328P Datasheet,' Microchip Technology Inc., 2018. [En línea]. Disponible en: https://www.microchip.com
- ETSI, 'Digital Video Broadcasting (DVB); Framing structure, channel coding and modulation for digital terrestrial television,' EN 300 744, European Telecommunications Standards Institute, 2009. https://www.etsi.org/deliver/etsi_en/300700_300799/300744/01.06.01_60/en_300744v010601p.pdf

Lima, abril de 2026 | Semestre 2026-I | Sistemas Digitales